

0.91.7. Le snake — le A avance tout seul

Le 0.91.6, mais le A ne s'arrête plus : il avance tout seul, un pas toutes les 250 ms. La croix ne fait que choisir la direction. Comme un vrai snake.

1. Ce qu'il faut comprendre

C'est le 0.91.6, avec une chose en plus : le A avance tout seul. Même sans toucher à rien, il fait un pas toutes les 250 ms. La croix ne le fait plus avancer : elle **choisit sa direction**. C'est comme ça que bouge un vrai serpent, dans un vrai snake.

`deplace_croix` s'en va. Elle faisait un pas seulement quand on appuyait. On la remplace par deux morceaux écrits à la main : **choisir** la direction, puis **avancer**.

Ce qui est nouveau ici : retenir une direction dans un nombre. `uint8_t sens = 0;` Il y a quatre directions ; on leur donne un numéro : **0 = droite, 1 = bas, 2 = gauche, 3 = haut**. Le programme ne retient que ce numéro. Au départ, 0 : le A part vers la droite. `nouvellePartie()` le remet à 0.

Choisir : `if (bouton(DROITE)) sens = 0;` et de même pour les trois autres. Il suffit d'**appuyer une fois**, même très court : `sens` change, et **il le reste** quand on lâche. Rien ne remet `sens` à zéro : le A continue dans la dernière direction choisie.

Avancer : `if (chaque(250)) { ... }` (le 0.77) : quatre fois par seconde. `effacer(x, y, 1)` enlève le A de sa case. Puis **un seul** des quatre `if` est vrai, celui du `sens` : il change `x` ou `y` d'une case. Enfin `poser(x, y, ALPHABET[0])` dessine le A sur sa nouvelle case.

Pourquoi `+ 1` et `- 1` ? Les colonnes grandissent vers la droite : droite, c'est `x + 1` ; gauche, `x - 1`. Les lignes grandissent vers le **bas** (la ligne 0 est en haut) : bas, c'est `y + 1` ; haut, `y - 1`.

Deux `chaque()` dans le même programme ? Oui : `chaque(250)` pour les pas, `chaque(1000)` pour le chronomètre. Chacun a **son propre chronomètre** : ils ne se gênent pas.

Déroulons : A en (5, 8), `sens` = 0. 250 ms : `x` = 6. 250 ms : `x` = 7. On touche BAS : `sens` = 1. 250 ms : `y` = 9, `x` reste 7. Le A descend maintenant, tout seul, jusqu'à ce qu'on choisisse autre chose.

Le reste ne change pas : le mur fait reculer le A (le 0.91.3) ; il reste donc collé au mur tant qu'on ne choisit pas une autre direction. La queue le suit, le P le fait grandir (le 0.91.6). Le A doit maintenant être **posé** dans `nouvellePartie()` : `deplace_croix` le faisait à sa place.

Pour plus tard : en snake, on ne peut pas faire demi-tour d'un coup, sur sa propre queue ; et toucher un mur ou sa queue fait perdre.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <couleurTexte>` → [0.87.7. La fonction couleurTexte\(\) — la couleur des lettres](#)
- `#include <texteGrand>` → [0.86.3. La fonction texteGrand\(\) — un texte agrandi](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <hasard>` → [0.81.2. La fonction hasard\(\) — un nombre au hasard](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2548 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	104 o	← la console, à l'allumage — la mise en route de la console
texte_grand_0()	195 o	← main() — ta fonction
chaque_minuteur()	124 o	← main() — une fonction fournie par le langage
viderEcran()	33 o	← nouvellePartie(), main() — ta fonction
nouvellePartie()	179 o	← main() — ta fonction
main()	735 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
Reste	10 o	← main() — le reste d'une division
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Hasard	25 o	← hasard() — tire un nombre au hasard
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	848 o	← CopierTuiles — la police entière, et 9 tuiles dessinées
"SCORE"	5 o	← texte() — un texte gravé
"TEMPS"	5 o	← texte() — un texte gravé
"APPUIE SUR START"	16 o	← texte() — un texte gravé
"FIN"	3 o	← texte() — un texte gravé
"SCORE"	5 o	← texte() — un texte gravé
"START : REJOUER"	15 o	← texte() — un texte gravé
GRAND_TABLE_0_0_0	12 o	← poser() — une table gravée
GRAND_TABLE_0_1_1	12 o	← poser() — une table gravée
GRAND_TABLE_0_2_2	12 o	← poser() — une table gravée
GRAND_TABLE_0_3_3	12 o	← poser() — une table gravée

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser() × 16 291 o sur place, dans texte_grand_0(), nouvellePartie(), main() + AdresseCase (10 o), GRAND_TABLE_0_0_0 (12 o), GRAND_TABLE_0_1_1 (12 o), GRAND_TABLE_0_2_2 (12 o), GRAND_TABLE_0_3_3 (12 o)

texte()	× 6	66 o sur place, dans nouvellePartie(), main() + EcrireTexte (10 o)
effacer()	× 3	64 o sur place, dans viderEcran(), main() + AdresseCase (10 o), EffacerCases (12 o)
bouton()	× 6	30 o sur place, dans main() + LireManette (37 o)
nombre()	× 3	30 o sur place, dans main() + EcrireNombre (33 o)
couleurFond()	× 1	12 o sur place, dans main()
hasard()	× 2	6 o sur place, dans main() + Hasard (25 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()

Tes fonctions appelées :

chaque_minuteur()	× 2	21 o sur place, dans main()
viderEcran()	× 2	6 o sur place, dans nouvellePartie(), main()
nouvellePartie()	× 2	6 o sur place, dans main()
texte_grand_0()	× 1	3 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <reste> // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
#include <effacer> // efface des cases, ou tout le fond
#include <poser> // pose une tuile sur une case du fond
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
#include <texte> // écrit un texte à l'écran
#include <couleurTexte> // choisit la couleur des lettres
#include <texteGrand> // écrit un texte agrandi, de 1 à 20 fois
#include <bouton> // lit un bouton de la manette
#include <chaque> // répond 1 toutes les n millisecondes
#include <hasard> // tire un nombre au hasard
#include <nombre> // écrit un nombre en chiffres
```

2. UNE VARIABLE — ECRAN

```
uint8_t ecran = 0; // 0 = le TITRE, 1 = le JEU, 2 = la FIN
```

3. UNE VARIABLE — X

```
uint8_t x = 5; // le A : sa colonne...
```

4. UNE VARIABLE — Y

```
uint8_t y = 8; // ...et sa ligne (le 0.81)
```

5. UNE VARIABLE — PX

```
uint8_t px = 15; // la pièce, le P : sa colonne...
```

6. UNE VARIABLE — PY

```
uint8_t py = 8; // ...et sa ligne
```

7. UNE VARIABLE — SCORE

```
uint8_t score = 0; // les pièces ramassées
```

8. UNE VARIABLE — TEMPS

```
uint8_t temps = 30; // les secondes qui restent
```

9. UNE VARIABLE — AX

```
uint8_t ax = 5; // la place du A AVANT son pas... (le 0.91.3)
```

10. UNE VARIABLE — AY

```
uint8_t ay = 8;           // ...pour l'y remettre s'il entre dans un mur
```

11. UN TABLEAU — QX

```
uint8_t qx[50];           // la queue (le 0.91.6), un TABLEAU de 50 cases 0 : les colonnes...
```

12. UN TABLEAU — QY

```
uint8_t qy[50];           // ...et les lignes. qx[0] : la case collée au A.
```

13. UNE VARIABLE — LONGUEUR

```
uint8_t longueur = 0;      // combien de cases la queue utilise (0 : le A seul)
```

14. UNE VARIABLE — SENS

```
uint8_t sens = 0;          // NOUVEAU : où va le A. 0 = droite, 1 = bas, 2 = gauche, 3 = haut
```

15. UNE FONCTION — VIDERECRAN()

```
void viderEcran() {  
    for (uint8_t l = 0; l < 18; l++) {  
        effacer(0, l, 20);  
    }  
}  
  
// ---- Une partie qui commence, TOUT au départ (le 0.91.2) ----  
// Depuis le titre, ET depuis la fin (START : rejouer).
```

16. UNE FONCTION — NOUVELLEPARTIE()

```
void nouvellePartie() {
    x = 5;                                // le A revient à sa place
    y = 8;
    px = 15;                             // le P revient en (15, 8), d'où qu'il soit
    py = 8;
    ax = 5;                               // et sa place d'avant, la même
    ay = 8;
    longueur = 0;                         // pas de queue, le A est tout seul (le 0.91.6)
    sens = 0;                             // NOUVEAU : le A repart vers la droite
    score = 0;                            // pas encore de pièce
    temps = 30;                           // 30 secondes de nouveau
    ecran = 1;                            // on est sur le JEU
    viderEcran();
    poser(px, py, ALPHABET[15]);
    poser(x, y, ALPHABET[0]);             // NOUVEAU : le A (deplace_croix ne le pose plus)

    // ---- Le cadre de murs, des X tout autour du terrain (le 0.91.3) ----
    //
    // colonne : 0 1 2 ...                19
    // ligne 0 : X X X X X X ... X X X    le haut : ligne 0, colonnes 0 à 19
    // ligne 1 : X                        X
    // ...      X   A           P   X    les côtés : colonnes 0 et 19
    // ligne 15: X                        X
    // ligne 16: X X X X X X ... X X X    le bas : ligne 16
    // ligne 17: SCORE 000  TEMPS 030    la ligne 17 reste aux nombres
    //
    // ALPHABET[23] : la 24e lettre (on compte depuis 0), le X.
    for (uint8_t c = 0; c < 20; c++) {
        poser(c, 0, ALPHABET[23]);       // le haut
        poser(c, 16, ALPHABET[23]);      // le bas
    }
    for (uint8_t l = 1; l < 16; l++) {
        poser(0, l, ALPHABET[23]);        // le côté gauche
        poser(19, l, ALPHABET[23]);       // le côté droit
    }

    texte(0, 17, "SCORE");
    texte(11, 17, "TEMPS");
}
```

17. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    couleurTexte(31, 16, 0);
    texteGrand(4, 5, "JEU", 4);
    texte(2, 12, "APPUIE SUR START");

    while (true) {
        image();

        if (ecran == 0 && bouton(START)) {
            nouvellePartie();          // la première partie
        }

        if (ecran == 1) {
            // ---- NOUVEAU : la croix ne fait plus avancer, elle CHOISIT la direction ----
            // On appuie une fois, même très court : sens change, et il le reste.
            if (bouton(DROITE)) sens = 0;
            if (bouton(BAS))    sens = 1;
            if (bouton(GAUCHE)) sens = 2;
            if (bouton(HAUT))   sens = 3;

            ax = x;              // on retient où est le A.. (le 0.91.3)
            ay = y;

            // ---- NOUVEAU : le A avance TOUT SEUL, un pas toutes les 250 ms ----
            // chaque(250) : vrai 4 fois par seconde (le 0.77). Ce chronomètre-ci
            // n'est pas celui de chaque(1000), plus bas : chacun a le sien.
            //
            //   sens :    3
            //               ↑
            //   2 ← A → 0   colonne : x - 1 à gauche, x + 1 à droite
            //               ↓
            //               1
            //   ligne   : y - 1 en haut, y + 1 en bas
            if (chaque(250)) {
                effacer(x, y, 1);          // le A quitte sa case
                if (sens == 0) x = x + 1;  // à droite
                if (sens == 1) y = y + 1;  // en bas
                if (sens == 2) x = x - 1;  // à gauche
                if (sens == 3) y = y - 1;  // en haut
                poser(x, y, ALPHABET[0]);  // et se pose sur la suivante
            }

            // ...et s'il est arrivé SUR le cadre (colonne 0 ou 19, ligne 0 ou 16) :
            // 1. on remet le X, que le A venait de recouvrir ;
            // 2. le A revient à sa place d'avant, et s'y pose.
            // || veut dire « OU » : une seule des quatre suffit.
            if (x == 0 || x == 19 || y == 0 || y >= 16) {
                poser(x, y, ALPHABET[23]); // le mur revient
                x = ax;                     // le A recule
                y = ay;
                poser(x, y, ALPHABET[0]);
            }

            // ---- La queue suit le A (le 0.91.5) ----
            // Le A a-t-il bougé ? Sa place n'est plus celle d'avant (ax, ay).
            // longueur > 0 d'abord (le 0.91.6). Sans queue, rien à déplacer, et
            // longueur - 1 vaudrait 255 (un uint8_t ne descend pas sous 0).
            // && veut dire « ET » : il faut une queue ET que le A ait bougé.
            if (longueur > 0 && (x != ax || y != ay)) {
                // 1. le BOUT de la queue disparaît : la dernière case, longueur - 1
                effacer(qx[longueur - 1], qy[longueur - 1], 1);

                // ---- 2. chaque case prend la place de celle de devant ----
                // En partant du BOUT, et en reculant (i-- : i = i - 1).
                // Avec longueur = 3 : i = 2 : qx[2] = qx[1]
```

```

//          i = 1 : qx[1] = qx[0]
//          i = 0 : 0 > 0 est faux, la boucle s'arrête.
for (uint8_t i = longueur - 1; i > 0; i--) {
    qx[i] = qx[i - 1];
    qy[i] = qy[i - 1];
}

// 3. la première case prend la case que le A vient de quitter
qx[0] = ax;
qy[0] = ay;

// 4. toute la queue redessinée, de la case 0 à la case longueur - 1
// (après une pièce, la nouvelle case venait d'être effacée avec le bout)
for (uint8_t i = 0; i < longueur; i++) {
    poser(qx[i], qy[i], ALPHABET[14]);
}
poser(x, y, ALPHABET[0]);          // 5. le A par-dessus (le 0.91.5)
}
if (x == px && y == py) {          // le A sur le P : ramassée (le 0.81)
    score = score + 1;

// ---- La pièce réapparaît AU HASARD, mais DANS le cadre (le 0.91.4) ----
//
//   px = 1 + hasard() % 18;
//   |   |   |
//   |   |   +-- % 18 : le reste de la division par 18, de 0 à 17.
//   |   |   Exemple : hasard() rend 137 ; 137 = 7 × 18 + 11 ;
//   |   |   le reste est 11.
//   |   +----- un nombre imprévisible, de 0 à 255 (le 0.81.2)
//   +----- + 1 : on décale de 0...17 à 1...18 → 11 + 1 = 12.
//
// Pourquoi pas % 20 comme au 0.81.2 ? Les colonnes 0 et 19 sont des murs X :
// la pièce doit rester de 1 à 18. Les lignes 0 et 16 aussi sont des murs :
// la pièce doit rester de 1 à 15, d'où 1 + hasard() % 15.
px = 1 + hasard() % 18;          // une colonne : 1 à 18
py = 1 + hasard() % 15;          // une ligne : 1 à 15
poser(px, py, ALPHABET[15]);     // le nouveau P apparaît là

// ---- La queue grandit d'une case (le 0.91.6) ----
// La nouvelle case naît SUR le bout de la queue. Au prochain pas, toutes
// les autres avancent, elle non : elle reste derrière.
// La toute première n'a pas de bout : elle naît SOUS le A.
// < 50 : le tableau n'a que 50 cases, qx[0] à qx[49].
if (longueur < 50) {
    if (longueur == 0) {
        qx[0] = x;                // la première case : sous le A
        qy[0] = y;
    } else {
        qx[longueur] = qx[longueur - 1]; // la case d'après le bout...
        qy[longueur] = qy[longueur - 1]; // ...à la même place que le bout
    }
    longueur = longueur + 1;       // une case de plus
}
}
nombre(6, 17, score);
nombre(17, 17, temps);
}

if (ecran == 1 && chaque(1000)) {
    temps = temps - 1;
    if (temps == 0) {
        ecran = 2;
        viderEcran();
        texte(8, 6, "FIN");
        texte(5, 9, "SCORE");      // le score de la partie (le 0.91.2)
        nombre(11, 9, score);
    }
}

```

```

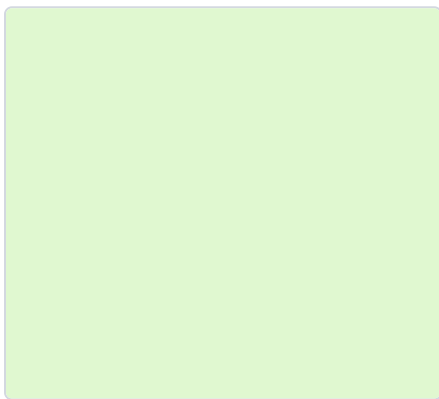
    texte(2, 13, "START : REJOUER");
  }
}

// ---- Sur la fin, START relance une partie (le 0.91.2).
if (ecran == 2 && bouton(START)) {
  nouvellePartie();          // la même fonction : tout au départ
}
}
}

```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran n'a pas bougé



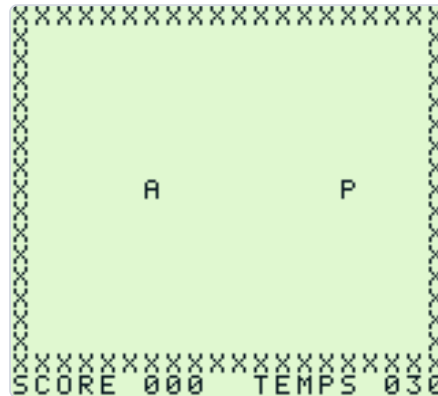
GAUCHE — l'écran n'a pas bougé



HAUT — l'écran n'a pas bougé



BAS — l'écran n'a pas bougé



START — l'écran a changé

5. Ce qu'on doit voir

START : le A part tout seul vers la droite ; un appui sur la croix change sa direction, et il continue sans s'arrêter.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ sans rien toucher, le A avance à droite ($x : 5 \rightarrow 9$)
- ✓ un appui court sur BAS : il descend, et continue tout seul ($x = 9, y = 13$)

Ensuite : **0.91.8. Le snake — le mur fait perdre** — chapitre 0 : Avant tout. · `gameboy3` — écrire une cartouche Game Boy en C++